

a slightly panicked guide to python, data, and machine learning

Marcus Ong

05 May, 2026

This guide, as you know, wasn't written by a professor or someone with a fancy AI researcher job. It was written by me, someone who spent a lot of late nights watching youtube tutorials, googling stuff, and slowly figuring this stuff out on my own. i'm not the best at this. i'll be honest about that. but i genuinely love it.

Table of contents

Course Description	1
Chapter 1 — Python Basics	2
Variables & Data Types	2
Arithmetic & Comparison Operators	4
Lists & Dictionaries	5
Lists	5
Dictionaries	6
Conditionals	7
Loops	8
for loops	8
Functions	9
Key Takeaways	12

Course Description

This course explores the main debates on the causes and consequences of political violence. We will focus on three major topics: civil wars, state-sponsored violence, and terrorism. Since the end of World War II, domestic conflict has largely outpaced international wars as the dominant type of violence. But what makes civil wars so prevalent in recent years? What are

the conditions under which a state decides to attack its own citizens? Why some groups resort to terrorism while others prefer nonviolent tactics?

The class has three goals. First, students will become familiar with the literature on political violence, its most important debates and recent findings. Second, students should be able to evaluate research methods and critically assess distinct theoretical approaches. Lastly, the course will develop the students' writing skills by asking them to review academic articles and present a paper of their own by the end of the semester.

Chapter 1 — Python Basics

What You'll Learn

- How Python stores and works with different kinds of data
- How to do calculations and make comparisons
- How to organise data in lists and dictionaries
- How to make decisions and repeat actions in your code
- How to write reusable functions

I want to be upfront about something. A lot of Python tutorials start with “Hello, World!” — you write one line, it prints a greeting, everyone claps. It's fine. But it doesn't really tell you anything useful.

So we're skipping it. This chapter covers the stuff that actually matters for working with data. Not everything Python can do — just the parts you'll use constantly.

Variables & Data Types

Before Python can do anything useful, it needs to store information. That's what variables are for.

```
name = "Marcus"  
age = 22  
gpa = 3.7  
enrolled = True
```

Each line creates a variable — a name that points to a value. Python figures out what *type* of value it is automatically.

```
print(type(name))
print(type(age))
print(type(gpa))
print(type(enrolled))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Here's what each type means in plain English:

- **str** — text, always wrapped in quotes. "Marcus", "hello", "42" (note: "42" is text, not a number)
- **int** — a whole number. 22, 0, -5
- **float** — a number with a decimal. 3.7, 0.5, 100.0
- **bool** — either **True** or **False**, nothing else. Used for yes/no decisions

Type matters because Python treats them differently. You can multiply two floats. You can't multiply a float and a string — at least not in the way you'd expect.

```
print(gpa * 2)      # 7.4 - works fine
print(name * 2)     # "MarcusMarcus" - technically works, but rarely what you want
print(gpa + name)   # TypeError - Python refuses
```

When you need to mix types, convert explicitly:

```
print("My GPA is " + str(gpa))
```

My GPA is 3.7

! Important

The point is: Every value in Python has a type. Knowing the type tells you what you can and can't do with it.

Arithmetic & Comparison Operators

Python handles the usual arithmetic you'd expect:

```
a = 10
b = 3

print(a + b)    # 13 - addition
print(a - b)    # 7  - subtraction
print(a * b)    # 30 - multiplication
print(a / b)    # 3.3333... - division (always returns a float)
print(a // b)   # 3  - floor division (rounds down)
print(a % b)    # 1  - modulo (the remainder)
print(a ** b)   # 1000 - exponentiation (10 to the power of 3)
```

The ones that trip people up are `//` and `%`. Floor division chops off the decimal. Modulo gives you what's left over after dividing. Both come up more than you'd think in data work.

Comparison operators return `True` or `False`. You'll use these constantly in conditionals and filtering data:

```
print(10 > 3)    # True
print(10 < 3)    # False
print(10 == 10)  # True - note the double equals
print(10 != 3)   # True - not equal
print(10 >= 10)  # True
print(10 <= 9)   # False
```

Warning

`=` and `==` are not the same thing. `=` assigns a value. `==` checks if two things are equal. Using `=` when you mean `==` is one of the most common bugs beginners write — and Python won't always catch it for you.

You can also combine conditions with `and`, `or`, and `not`:

```
age = 22
enrolled = True

print(age > 18 and enrolled)    # True - both must be true
print(age > 18 or enrolled)     # True - at least one must be true
print(not enrolled)             # False - flips the value
```

! Important

The point is: Arithmetic operators do maths. Comparison operators ask questions and return True or False. You'll use both constantly.

Lists & Dictionaries

Real data is almost never a single value. It's a collection of values. Python has two main ways to store collections, and they work very differently.

Lists

A list is an ordered collection. The order matters, and items can repeat.

```
grades = [85, 90, 78, 92, 65]
print(grades)
```

```
[85, 90, 78, 92, 65]
```

You access items by their position, called an **index**. Indexing starts at 0, not 1.

```
print(grades[0])    # 85 - first item
print(grades[1])    # 90 - second item
print(grades[-1])   # 65 - last item (negative indexes count from the end)
```

Some useful things you can do with lists:

```
grades.append(88)    # add an item to the end
grades.remove(78)    # remove a specific value
print(len(grades))   # how many items are in the list
print(sum(grades))   # add them all up
print(min(grades))   # smallest value
print(max(grades))   # largest value
```

Dictionaries

A dictionary stores **key-value pairs**. Instead of a position, each value has a label.

```
student = {  
    "name": "Marcus",  
    "age": 22,  
    "gpa": 3.7,  
    "enrolled": True  
}
```

You access values by their key:

```
print(student["name"])    # Marcus  
print(student["gpa"])    # 3.7
```

You can add or update entries the same way:

```
student["year"] = 3        # adds a new key  
student["gpa"] = 3.9       # updates an existing one
```

And you can check what keys exist:

```
print(student.keys())  
print(student.values())
```

```
dict_keys(['name', 'age', 'gpa', 'enrolled', 'year'])  
dict_values(['Marcus', 22, 3.9, True, 3])
```

Tip

In data science, you'll often see a list of dictionaries — one dictionary per row of data, collected into a list. This is basically a table, and it's exactly the structure that pandas (Chapter 2) is built around.

Important

The point is: Lists store ordered collections you access by position. Dictionaries store labelled data you access by key. Most real data is one or the other.

Conditionals

Conditionals let your code take different paths depending on whether something is true or false.

```
gpa = 3.7

if gpa >= 3.5:
    print("Dean's List")
elif gpa >= 2.0:
    print("Good Standing")
else:
    print("Academic Probation")
```

Dean's List

Python reads this top to bottom and stops at the first condition that's true. Everything under it is ignored.

The indentation is not decoration. Python uses it to know what belongs inside each block. Four spaces is the standard.

You can nest conditionals inside each other, though it gets hard to read quickly:

```
gpa = 3.7
enrolled = True

if enrolled:
    if gpa >= 3.5:
        print("Enrolled and on Dean's List")
    else:
        print("Enrolled but not on Dean's List")
else:
    print("Not enrolled")
```

Enrolled and on Dean's List

! Important

The point is: `if` checks a condition. `elif` checks another if the first was false. `else` catches everything that didn't match. Python stops at the first true one.

Loops

A loop runs the same block of code multiple times. In data science, you'll use loops to process every row in a dataset, every column in a table, every file in a folder.

for loops

```
grades = [85, 90, 78, 92, 65]

for grade in grades:
    print(grade)
```

```
85
90
78
92
65
```

`for grade in grades` means: take each item from `grades`, temporarily call it `grade`, and run the indented block with it. When the list runs out, the loop stops.

You can also loop over a range of numbers:

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

`range(5)` generates the numbers 0 through 4. You can customise it:

```
range(1, 6)      # 1, 2, 3, 4, 5 - start and stop
range(0, 10, 2)  # 0, 2, 4, 6, 8 - start, stop, step
```

A common pattern is building a new list inside a loop:


```
grades = [85, 90, 78, 92, 65]
curved = []

for grade in grades:
    curved.append(grade + 5)

print(curved)
```

[90, 95, 83, 97, 70]

We started with an empty list, then added each adjusted grade one at a time.

💡 Tip

List comprehensions are a more compact way to write this pattern, and you'll see them everywhere in data science code:

```
curved = [grade + 5 for grade in grades]
```

Same result, one line. Worth knowing when you see it in the wild.

❗ Important

The point is: for loops repeat a block of code for each item in a collection. `range()` lets you loop a specific number of times.

Functions

A function is a named, reusable block of code. You define it once, then call it whenever you need it.

```
def calculate_average(grades):
    total = sum(grades)
    count = len(grades)
    return total / count
```

Now let's use it:

```
my_grades = [85, 90, 78, 92, 65]
print(calculate_average(my_grades))
```

82.0

`def` starts the definition. The name comes next. Inside the parentheses are **parameters** — placeholders for the values you'll pass in. `return` sends a result back to wherever the function was called.

Functions can take multiple parameters:

```
def weighted_average(grades, weights):
    total = sum(g * w for g, w in zip(grades, weights))
    return total / sum(weights)

grades = [85, 90, 78]
weights = [0.2, 0.3, 0.5]

print(weighted_average(grades, weights))
```

82.5

You can also set **default values** for parameters, so the function works even if you don't pass everything in:

```
def describe_grade(score, passing=50):
    if score >= passing:
        return "Pass"
    else:
        return "Fail"

print(describe_grade(74))          # uses default passing=50
print(describe_grade(74, 75))     # overrides the default
```

Pass

Fail

! Important

The point is: Functions let you write a piece of logic once and reuse it as many times as you need. They take inputs through parameters and send results back with `return`.

i Python vs Excel — Same Job, Different Tool

If you've used Excel, you've already done programming. You just didn't call it that.

Excel	Python
Cell with a value	Variable
<code>=A1 + B1</code>	<code>a + b</code>
<code>=IF(A1>50, "Pass", "Fail")</code>	<code>if score > 50:</code>
Dragging a formula down a column	<code>for</code> loop
<code>=AVERAGE(A1:A10)</code>	<code>sum(grades) / len(grades)</code>
Named range	List or dictionary

The difference is scale. A spreadsheet with 500,000 rows gets slow, crashes, and becomes impossible to audit. A Python script handles it without blinking, and you can read exactly what it did. Excel is great for things you want to see and click around in. Python is better for things you want to automate and repeat.

! Common Beginner Mistakes

Forgetting the colon. `if score > 50` will throw a `SyntaxError`. It needs to be `if score > 50:`. Same with `for`, `def`, and anything else that opens a block.

Mixing up `=` and `==`. `=` assigns a value. `==` checks equality. `if grade = 90` is a syntax error. `if grade == 90` is what you want.

Index starts at 0. `grades[0]` is the first item. `grades[1]` is the second. Trying to access `grades[5]` on a list of 5 items will crash — the last valid index is `grades[4]`.

Mutable default arguments. Don't use a list as a default parameter value in a function. It's a Python quirk that causes strange bugs. Use `None` and create the list inside the function instead.

Key Takeaways

- Variables store values; data types (str, int, float, bool) tell Python what kind of value it is
 - Arithmetic operators do maths; comparison operators return True or False
 - Lists store ordered collections accessed by index; dictionaries store labelled data accessed by key
 - Conditionals let code take different paths; loops repeat a block for each item in a collection
 - Functions package up logic so you can reuse it — they take inputs and return outputs
-

Try It Yourself

Write a function called `summarise` that takes a list of numbers and prints the count, sum, minimum, maximum, and average. Then call it on a list of at least 8 values. If you want to go further, add a parameter that lets the caller choose whether to print the results or return them as a dictionary.

Ready for Chapter 2 — Working with Data?